32.2.1 位置实参

你调用函数时, Lua 必须将函数调用中的每个实参都关联到函数定义中的一个形参。下面我们来看一个显示武将信息的函数:

```
local describe
```

Part V

制作 Lua 拓展

CHAPTER 33

开发环境准备

33.1 通过本教程，你可以做到…

本教程将讲解如何制作新月杀扩展，包括武将、游戏牌和模式等。我们在新月杀里玩到的武将、游戏牌和模式等，主要都是用计算机语言 Lua 编写完成的。本教程就将介绍如何为新月杀制作 Lua 扩展。所有新月杀已有的武将、游戏牌和模式都可以作为我们的参考。

33.2 熟悉新月杀

在制作扩展前，我们先需要熟悉一下新月杀的文件结构。

打开新月杀文件目录，和制作扩展相关的主要有 packages、lua 和 docs 这三个文件夹。

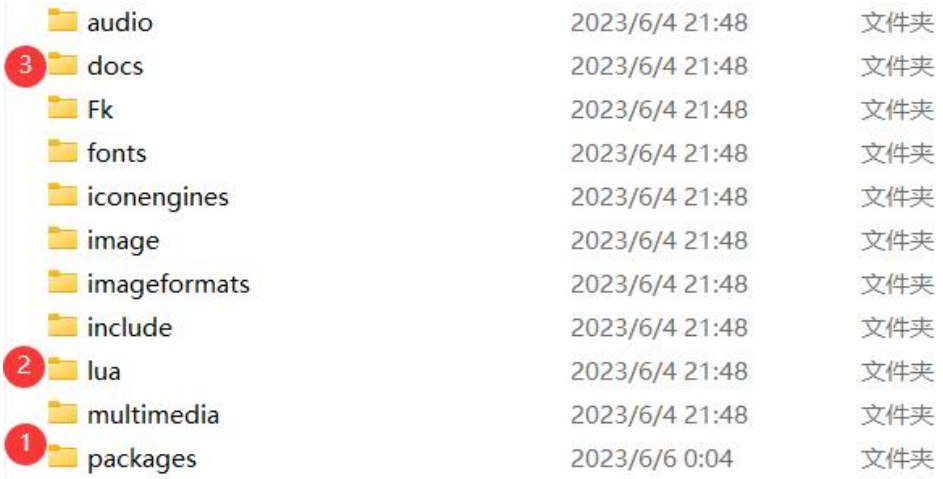


图 1: 新月杀的目录结构

33.2.1 packages/

packages 文件夹是储存所有游戏卡牌扩展和武将扩展的位置。新月杀的本体只包含 standard、standard_cards、maneuvering 以及 test 四个包，它们属于核心扩展，也就是三国杀的标准包（武将）、标准游戏牌包和军争游戏牌包，还有（专门用于测试而非游玩的）测试包。

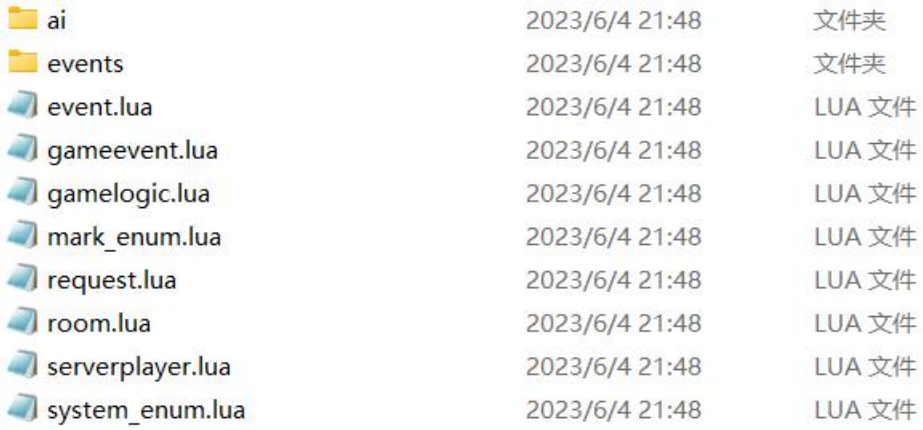
如果你进行过服务器联机的话，会自动同步服务器上的所有拓展包，例如 shzl（神话再临）。在新月杀中，扩展包拥有在线安装/在线更新的功能，这种功能都是依托于 git 进行的，因此如果你打算将自己的扩展包发布出去的话，就需要为其创建 git 仓库，并托管到 git 托管网站去。如何进行扩展包发布和 git 托管参见其他教程。

图 2: packages/下存放着拓展包

提示: 如果你没有进行过联机，可以直接在新月杀的管理扩展包功能中，使用“从 URL 安装”，输入以下扩展包的链接，将这些本教程会涉及到的需参考的扩展包安装入你的新月杀：神话再临、一将成名、界限突破标包。

33.2.2 lua/

lua 文件夹是新月杀运行规则的核心。我们以 lua 文件夹下的 server 文件夹为例：



ai	2023/6/4 21:48	文件夹
events	2023/6/4 21:48	文件夹
event.lua	2023/6/4 21:48	LUA 文件
gameevent.lua	2023/6/4 21:48	LUA 文件
gamelogic.lua	2023/6/4 21:48	LUA 文件
mark_enum.lua	2023/6/4 21:48	LUA 文件
request.lua	2023/6/4 21:48	LUA 文件
room.lua	2023/6/4 21:48	LUA 文件
serverplayer.lua	2023/6/4 21:48	LUA 文件
system_enum.lua	2023/6/4 21:48	LUA 文件

图 3: server 文件夹

server 文件夹中包含许多在客户端执行文件。例如，events 文件夹中是三国杀各类“事件”的执行规则，它定义着诸如使用牌事件的流程、扣减体力事件各种时机的先后顺序，这些流程是我们按照规则集现有结算规则严格制定的。

虽然 lua 文件夹神通广大，但并非是我们扩展包开发需要修改的内容。我们需要熟悉其中游戏结算的逻辑，在其中查阅要利用的函数，寻找调试修正 bug 的方法。在没有思路实现某个功能时，翻翻 lua 文件夹总能有新发现 ^_^。

33.2.3 docs/

开发之初计划编写的文档，现已被新月之书取代，请无视吧。

33.3 准备代码编辑器

新月杀扩展的开发完全使用 Lua 语言。采用 Lua 不需要编译，语法简单易懂，外加和太阳神三国杀的一些渊源，使用 Lua 完全符合我们自由、开放、可扩展的 diy 平台理念。

尽管使用记事本就能直接编辑 Lua 文件，但工欲善其事必先利其器，用一款好的代码编辑器对于提高编程效率、查漏补缺是很有帮助的。出于易用性和免费的考虑，我们推荐用 VSCode 进行编写（百度搜索下载即可）。

图 4: vscode

VSCode 提供的插件能为开发提供很大便利。点击侧边栏，选择第五项“扩展”选项卡，直接搜索插件名字安装即可。

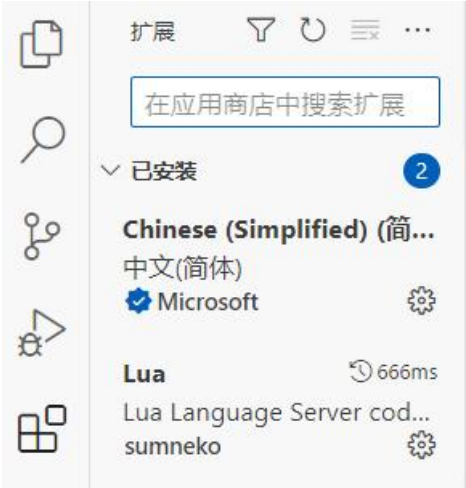


图 5: 插件面板

1. 中文翻译包

图 6: 安装中文翻译插件吧！

难道有人看英文比中文更顺眼？

2. Lua

图 7: 安装 Lua 插件吧！

Lua 插件提供了强大的代码提示、补齐、纠错、注释功能，妈妈再也不用担心我打错字啦 ^_^。

33.4 在 VSCode 中部署工作区

重要: 注意，因为我们开发、调试都是在本地进行的，还没有上传至服务器，所以最好复制一份游戏本体，用来玩的和用来开发的区分开，不然我们修改过文件的游戏是不能连接服务器的。将 FreeKill-release 文件夹（或者，你的新月杀的根目录所在的文件夹）复制一份，粘贴在另外一处，改名为 FreeKill-dev（或者“新月杀开发版”，名字无所谓啦）以区分。

为了方便在文件间进行切换，以及全局搜索相关内容，我们先将游戏目录部署到工作区。点击 VSCode 的侧边栏，选择第一项“资源管理器”选项卡：

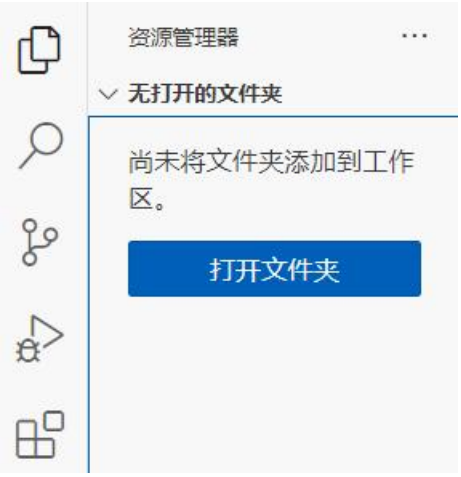


图 8: 资源管理器面板

点击“打开文件夹”，选择我们用来开发的游戏目录（FreeKill-dev 文件夹）。

资源管理器中显示出文件目录就部署好啦。

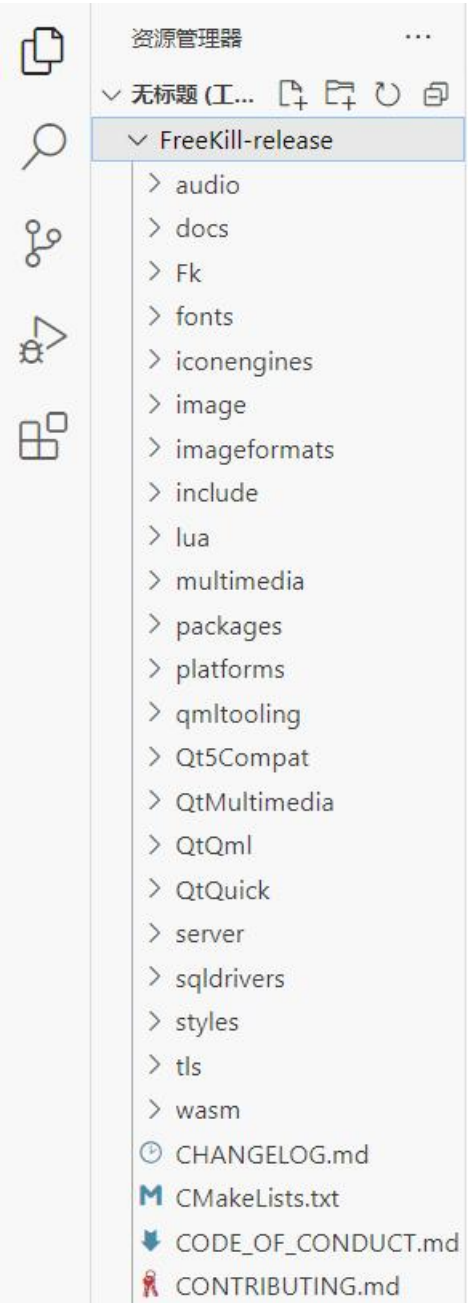


图 9: 部署完成工作区后，应该像这样

33.5 用 VSCode 写东西

好了，我们终于可以开写了……但是写啥呢……？你可以直接前往下一章节创建第一个武将，但这里先说一下你会碰到的操作/快捷键吧。

- `Ctrl+F` 是在**文件内**搜索特定字符串的常见快捷键，

  但是更常用的，则是它的高级形式——在整个工作区（也就是 FreeKill-dev 文件夹）里搜索，这个的快捷键是 `Ctrl+Shift+F`

  通过搜索已有的相对成熟的代码，可以解决很多问题 ~

- `Ctrl+S` 是保存本文件，你会经常摁它的。

  未保存的文件，其文件名左边会有一个实心圆，表示该文件有未保存的东西。

提示: 经常保存有益身心健康，谁也不想辛辛苦苦写的东西全作古吧……

- 左侧的资源管理器通常有四个区域——这里只摘选经常用的地方
  - 打开的编辑器：这里是你打开的所有文件的列表，未保存的文件也会显示一个实心圆。
  - 资源视图：其实名字一般是你的工作区（一般是游戏目录，不会有人拿扩展包当工作区吧），这里会用树形结构显示你打开的全部文件，也可以通过 `Ctrl+F` 按照**文件名**搜索文件。
  - 时间线：这里保存着你各种保存后的文件历史，点击其中一个历史记录会显示它与现存版本之间的差异。

  提示: 单纯撤销文件的历史的名字是不一样的，可以通过这个撤销你的撤销。

CHAPTER 34

创建拓展包与新武将

34.1 扩展包的内容

如在上一章所述，新月杀的游戏扩展文件全部位于 packages 文件夹。我们先看看游戏中现有的扩展是怎样的。打开 packages/shzl（神话再临）：

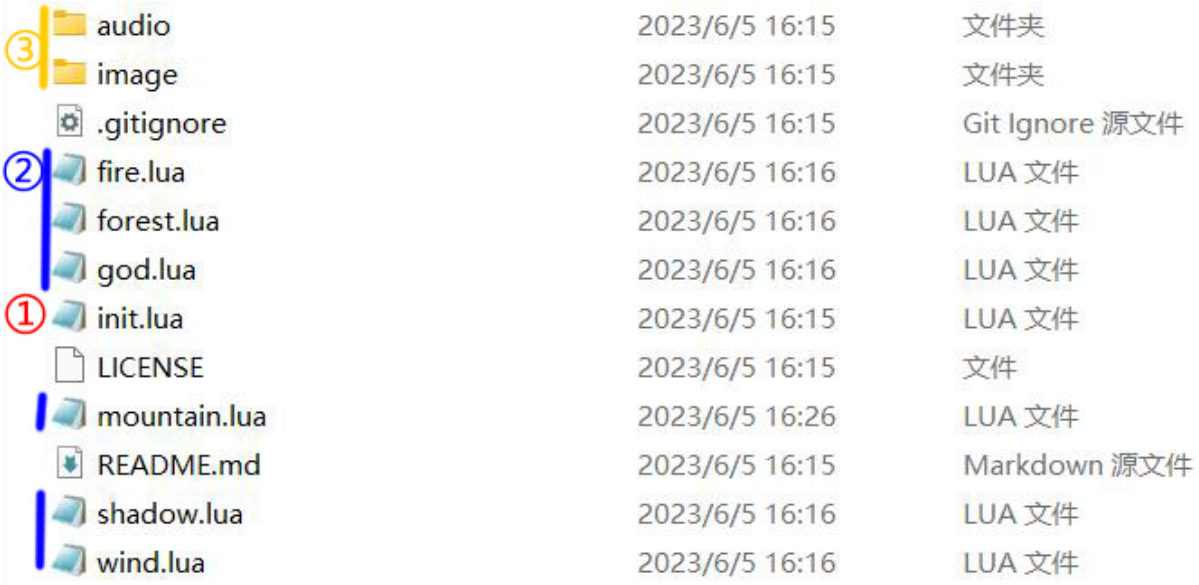


图 1: 拓展包 shzl 的内部文件

34.1.1 init.lua

init.lua 文件是扩展包的核心，严格意义上，扩展包只会加载 init.lua，所以扩展中其他所有的 lua 文件都需要通过 init.lua 来加载。打开 shzl 的 init.lua：

```
1  local wind = require "packages/shzl/wind"
2  local fire = require "packages/shzl/fire"
3  local forest = require "packages/shzl/forest"
4  local mountain = require "packages/shzl/mountain"
5  local shadow = require "packages/shzl/shadow"
6  local thunder = require "packages/shzl/thunder"
7  local god = require "packages/shzl/god"
8
9  return {
10   wind,
11   fire,
12   forest,
13   mountain,
14   shadow,
15   thunder,
16   god,
17 }
```

我们可以看到通过 `require`，对应目录下的 wind 风、fire 火等子扩展包都加载进来了。同理，也可以像标准包一样 `require` 要用的翻译文件、辅助函数。总而言之，只有通过 `require` 才能加载 init 以外的 lua。

我们简单地解析一下这几行代码的作用，以第 1 行为例：

```
local wind = require "packages/shzl/wind"
```

这行代码的意思是“定义一个局部变量——**wind**，然后将 `require` 的值赋给 **wind**”，

- **wind** 前面的 `local` 代表的就是它是 **局部变量**，没有了 `local`，它就是 **全局变量**。全局变量和局部变量都是 **变量**。变量在使用前，需要在代码中进行 **声明**，即创建该变量；在 lua 里面，`local a`，就是创建了一个局部变量 `a`。变量可以被访问，我们可以拿他来参与运算，也可以再赋值。一般而言，要 **慎用全局变量**，因为它们很容易相互打架，造成污染。全局变量会被编辑器标为红色。
- `=` 便是编程中最常见的 **赋值**符号，代表将右边的东西赋给左边。（可不是相等的意思，相等是 `==` ）在 lua 里面，`local a = 5` ，就是创建了一个局部变量 `a` 并用 5 为其赋值。
- `require` 则是本行代码最重要的部分，它返回之后紧跟着的双引号内的 lua 文件所代表的全部东西，什么都有。下面几行都是相同的道理，分别载入了风火林山阴雷神 7 个子扩展包，每个文件都有各个子扩展包的武将，例如曹仁、张角就在 **packages/shzl/wind.lua** 文件里面。
- init.lua 或者别的 lua 的最后都会有一个 `return` ，意思是返回整个模块，代表对整个文档的总结。这里返回了我们在上方定义的所有引用，并用 `{}` 包成了一个整体。第 10 行的 `wind` 就指向第 1 行声明过了的 `wind` 。局部变量在声明完毕之后，引用的时候就不用 `local` 啦（不然就变成再创建一个局部变量了）。

每个扩展包 lua 都需要 `return` 一个扩展包，否则这个扩展包就是不存在的。

提示： `require` 会将全局变量也加载进来，因此代码中要慎用全局变量。

提示： 当然也可以在 init.lua 塞进扩展包全部的内容，standard 标准包就是这样的。理论上这也可以，但是还是分开比较好，尤其是有下文所述限制的前提下。

34.1.2 子扩展包

一个大的扩展包系列会含有多个小扩展包，例如神话再临的风林火山，一将成名的 2011~……等。子扩展包是游戏中武将一览的划分区间。这样做既是为了分类美观、查找方便，也是限制单个 lua 文件大小。

提示： Lua 语言规定单个文件至多只能有 200 个局部变量，因此根据估算，我们建议一个子扩展包的武将数量不要超过 40。

34.1.3 audio, image

顾名思义是存放扩展包语音、图片的文件夹。对于我们的第一个扩展做得简单些，只需要了解：audio 中的 death 为武将阵亡语音，skill 为技能语音；image 中的 general 为武将图片。

34.1.4 其他文件

包括开源声明、git 托管文件等，先不管它们。

34.2 创建拓展包

我们模仿 shzl 的形式创建一个扩展。首先在 packages 文件夹下建立一个新文件夹。老传统，我们的第一个扩展就叫 study 学习包 ~

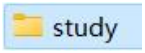


图 2: 新建文件夹

然后在 study 文件夹中创建一个 init.lua 文件。可以直接从其他包复制，也可以新建一个 txt 文本然后改名。用 VSCode 打开它，但在修改之前，请先稍微动一下手，点击右

下角这一块，将你的默认缩进改为空格:2，编码改为 UTF-8（默认是这个，但最好确认一下）

图 3: 注意编码必须是 UTF-8，空格 =2 只是推荐缩进

然后修改代码，加载子扩展包：

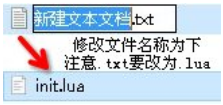


图 4: 新建文本文件并改名

```
1 local xuexi = require "packages/study/xuexi"
2
3 return {
4   xuexi,
5 }
```

提示:

1. 我们在代码中利用到的标点都是半角符号，也就是英文输入模式下的标点。比如第 1 行里的引号不是 “” 而是 "" ，第 4 行的逗号不是 ，而是 ，。（当然，通过设置输入法，我们也可以在中文输入模式下打出半角符号 ~）
2. 注意第 4 行开头的缩进。由于这一行属于下一个层级，我们需要往后缩进，让代码的结构更加清晰。在新月杀里，我们约定缩进 2 个空格。
3. 以 UTF-8 无 BOM 格式编码。
4. 所有文件尾必须留一个换行符，敲一个回车即可。（在这里的代码片段中，可能无法看到最后的空行，看示例代码更为清楚）

重要: 我们现在所做的一切操作结果都可以在 **引用示例.zip** 中找到，可以多多参考对照。

再在 study 文件夹下面建立一个新的 xuexi.lua，修改为：

```
local extension = Package:new("xuexi")
extension.extensionName = "study"

Fk:loadTranslationTable{
  ["xuexi"] = "学习",
}
```

（续下页）

(接上页)

```
7
8 return extension
```

- 1 行，声明以创建局部变量 extension，将其赋值为创建出的名为 xuexi 的扩展包。这里的名字是子扩展包的名字。接下来的代码中，extension 就将代表整个子扩展包。
- 2 行为这个子扩展包，也就是 extension ，指定 extensionName ，也就是它所属的大扩展包，其名字必须为我们文件夹的名字，整个大扩展包都是根据这个名字寻找路径的，包括 lua，包括图片，所以不要填错。
- 4~6 行是翻译表，这样在游戏中子扩展包的名字“xuexi”就会翻译为“学习”。
- 8 行返回扩展包，这样程序就知道我们这个子扩展包的代码到这里就结束啦。

这样我们的扩展包就创建完成了！~ 之后我们的代码操作都会在这个 **xuexi.lua** 文件之内，别跑到 *init.lua* 去啦。

图 5: 创建好的扩展文件和子扩展文件

保存好所有动过的文件。用 **单机启动**打开游戏，查看武将一览，可以看到已经出现我们的学习包了，不过现在还是空的没有武将。

图 6: 学习包！说起来 study 还是英文呢，要不自己加一句翻译?

> **注意:** 新的文件、修改过的代码文件都需要在 **保存**之后 **重新打开游戏**才能生效。我们开发新月杀一般都是在 **单机启动**环境下进行的。

34.3 创建武将

创建新武将的话，首先也是要先想好武将的名字。但是这件事情就简单多了，一般来说都是用 TA 的姓名的汉语拼音直接作为名字的。例如曹操就是 caocao，司马懿就是 simayi，当然，英文人物也可以用英文名，如某位凯撒大哥就是 caesar。

看起来武将的名字比扩展包的名字好写，但其实不然。要知道，在新月杀诞生的时候，别说界限突破了，谋攻篇都出来了，到了现在 (2024 年)，同名武将多到可以凑个军五甚至军八……所以对于这些同名武将，新月杀提供了一种方法来自动识别——

当你注册一个名字格式为 XXXX__YYYY（这里是双下划线，如 mou__caocao）的新武将时，系统会自动将这个名字沿中间切成两半：其中右边的 YYYY 是“武将真名”trueName，而左边的 XXXX 则是这个武将的特有标示，也就是“武将包名”，或称前缀。如果两个武将拥有同样的“武将真名”，那么这两个武将就是同名武将，例如 ex__caocao 和 mou__caocao 的“武将真名”都是 caocao，那么这两个武将就是同名武将。

按照传统，我们也该写个学习包专属的孙悟空的，但是……十周年和欢乐杀已经有孙悟空了 (悲)，所以在命名上我们也该规矩一点……意味着学习包的孙悟空名字最好改成 st__sunwukong 了，

这里前缀取的是”st”，study 嘛。在你自己的拓展包中，根据情况选定合适的前缀吧。

具体代码就是 local sunwukong = General:new(extension, "st__sunwukong", "god", 5)，之后你的 xuexi.lua 看起来就像这样：

重要： 要注意把 return extension 留在文档末尾，以下我们的操作都要在这一行 return extension 之前完成！

```lua
local extension = Package:new("xuexi")
extension.extensionName = "study"

Fk:loadTranslationTable{
  ["xuexi"] = "学习",
}

local sunwukong = General:new(extension, "st__sunwukong", "god", 5)

return extension
```

创建武将本质上是创建一个“**武将对象**”，为此，让我们看向**武将对象**的构造函数：

```lua
--- 构造函数，不可随意调用。
---@param package Package @ 武将所属包
---@param name string @ 武将名字
---@param kingdom string @ 武将所属势力
---@param hp integer @ 武将初始体力
---@param maxHp integer @ 武将初始最大体力
---@param gender Gender @ 武将性别
function General:initialize(package, name, kingdom, hp, maxHp, gender)
```

各参数的作用见上方，这里详细解释一下：

1. package 是该武将所属的扩展包，鉴于第一行已经定义了一个 extension，我们就用它了。
2. name 是这个武将的名字，就用刚才起的名字”st__sunwukong”吧。
3. kingdom 是这个武将所属的势力，按照传统，我们填个”god”，也就是神势力。
4. hp 和 maxHp 分别代表体力值和体力上限，maxHp 默认与 hp 一致，而 hp 默认为 4。
5. gender 就是武将性别，默认为 General.Male（男性）。

以上代码就往学习包里塞了一位名为“st__sunwukong”的 5 血 5 上限神势力男性武将 ~

至此武将的创建就完成了！但是如果我们现在进入游戏的话，他会显示为英文……那就用 Fk:loadTranslationTable 补一下翻译吧：

```
Fk:loadTranslationTable{
  ["xuexi"] = "学习",
  ["st"] = "学",
}

local sunwukong = General:new(extension, "st__sunwukong", "god", 5)
Fk:loadTranslationTable{
  ["st__sunwukong"] = "孙悟空",
  ["#st__sunwukong"] = "齐天大圣",
  ["designer:st__sunwukong"] = "设计者",
  ["cv:st__sunwukong"] = "配音演员",
  ["illustrator:st__sunwukong"] = "画师",
}
```

翻译表其实是个很纯粹的东西，你输入特定的字符串，就会根据翻译表输出对应的翻译字符。以下是武将信息里除了武将名以外的其他信息：

- "# 武将名" 是武将的称号，理论上要多长就有多长，不过考虑到实际显示，建议限制在 7 个字以内。
- "designer: 武将名" 是武将的设计者，默认为“官方”。
- "cv: 武将名" 是武将的配音演员，默认为“官方”。
- "illustrator: 武将名" 是武将的画师，默认为“官方”。

这些在进入游戏后的武将一览都能看到。

保存一下，到游戏里看看我们的孙悟空吧。打开武将一览-> 学习包：

图 7: 我们刚刚创建的悟空在此

好了，赶紧拿着你新写的这位武将去单挑吧！

PS：五体力白板孙悟空的单挑能力还是不弱的呢。^_^

CHAPTER 35

添加技能

白板武将虽然不算弱（其实已经很弱），但是游戏体验还是差了点。都 2024 年了，再端上五血白板多少有点欺负自己了……

解决方法？添加个技能呗，比如说……〖**无双**〗？正好标包有，也不用麻烦我们写了。

在 vscode 切到刚才的 xuexi.lua，在引入孙悟空的代码下面加个代码：`sunwukong:addSkill("wushuang")`

```
local sunwukong = General:new(extension, "st__sunwukong", "god", 5)
sunwukong:addSkill("wushuang")
```

就可以了。

备注： 在这里，像 addSkill 这样后面带括号的、可以帮我们完成一项功能的具体的方法，就是传说中的“函数”了。新月杀的绝大多数武将扩展行为，都是通过各种各样的函数分工合作，才能顺利完成的。（请回忆之前 Lua：函数一节）

至于 `General:addSkill()`，这个是真正赋予武将技能的函数，有了它，孙悟空才能拥有这个技能。该函数的原型如下：

```
--- 为武将增加技能，需要注意增加其他武将技能时的处理方式。
---@param skill Skill @ （单个）武将技能
function General:addSkill(skill)
```

显然，这个函数可以接受一个技能 `Skill`，也就是一个属于技能对象 `Skill` 的变量；但同时，这个函数也可以接受字符串，此时就可以添加别人的技能了 ~

注意武将名称（sunwukong）和函数名称（`addSkill` ）之间的那个符号，是冒号！

因为我们的武将名称 sunwukong 在之前那个 `General` 函数的引诱下已经成为了孙悟空

这个武将的形象代言人了，它代表了一个武将对象（General ）；而这个函数 addSkill 正是武将对象中的一个功能，我们称它为 General 的成员函数（亦称方法）。一个对象调用自己的成员函数的时候，代码里需要用冒号手拉手，来表示这种调用关系的。

无双的技能名字是 wushuang。一般地，技能的名字就是它的汉语拼音了，比如马术就是 mashu，奸雄就是 jianxiong，挑衅就是 tiaoxin。

当然也有例外，如曹操的护驾是 hujia，倚天蔡昭姬的胡笳则是 qyt___hujia，毕竟不能重名嘛。有关技能名字的信息，必要的时候还是要查看一下源代码的。

方法很简单，先点击左侧栏的放大镜开始全局搜索，然后将排除文件去掉（这样就能搜索所有的文件了），然后搜索我们想搜索的技能名就行了：

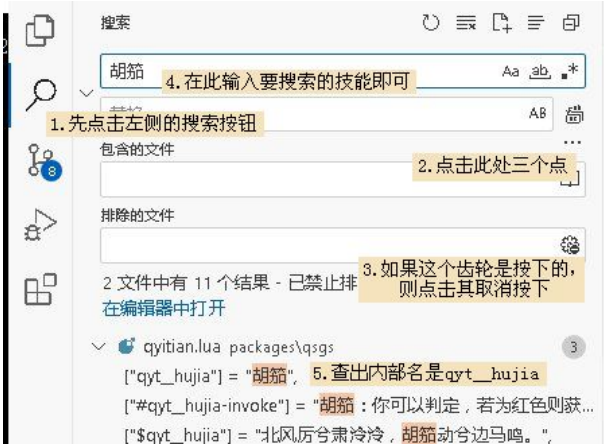


图 1: 借由全局搜索功能找到想要的技能名

保存一下，打开游戏，放到实际对局试试吧！

备注： 想稳定选到孙悟空的话，需要利用自由选将功能。创建房间之前启用自由选将，然后在选将时右键单击待选武将牌即可。

图 2: 会无双的悟空

这回总可以好好单挑了，五血无双还没负面技，总该……

CHAPTER 36

武将美化

武将有了技能，就该出来亮相了。但是总不能老让人家挂着貂蝉剪影乱跑吧？该有的美化工作还是要有的。武将的美化，在新月杀中只需要制作一张全身大图即可。

当然，美化之前问一问，做好准备了吗？Let’s go！

36.1 软件准备

我们这里介绍用 Gimp 来裁剪图片吧。Gimp 是一款挺有名气的免费开源图像处理软件哦。

前往官网下载吧：https://www.gimp.org/downloads/

官网提供了三种下载方式，这里我点击第二个“直接下载”。

Gimp 的安装也是很简单的，一路下一步就行了，视情况安装到 D 盘亦可。

36.2 图片准备

然后找一张图片做武将原画，上网搜一下吧。

要是找三国人物的话，现在三国杀的皮肤那么多，尽情选择吧。

孙悟空的话……还是去百度图片搜索找找看好了。

OK，总之准备就绪。

36.3 处理图片

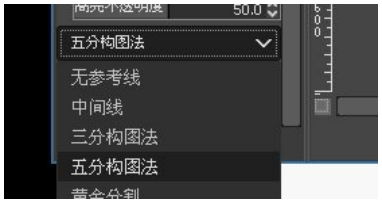
我们所需的武将大图，相关信息是这样的：

- 像素尺寸：250x292
- 图片格式：jpg 格式
- 存放位置：我们的拓展包/image/generals 下

嘛，首先我们先用 Gimp 打开图片，然后选择裁剪工具：

勾选固定宽高比，然后将宽高比设为 250:292，像这样：

然后再设置好参考线，待会我们要基于参考线裁剪图片的：



设置完了！开始切割图片吧。裁切的时候，注意让人物的头部位于第二行第三列那个格子左右的地方，这样就很有那种头部中等偏上的半身图感觉哦！

这样基本框定后，按下回车键，图片就裁切完了。但是现在还只是我们的图片是 250:292 而已，而尺寸要求是严丝合缝的 250x292 像素，所以我们还需要再把图片缩放一次。在图上右键单击，选择图像-> 缩放图像，开始修正尺寸。

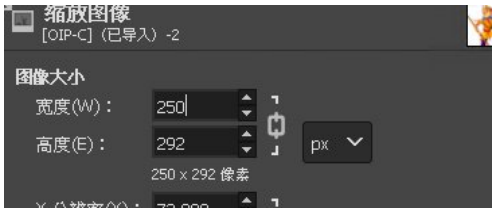
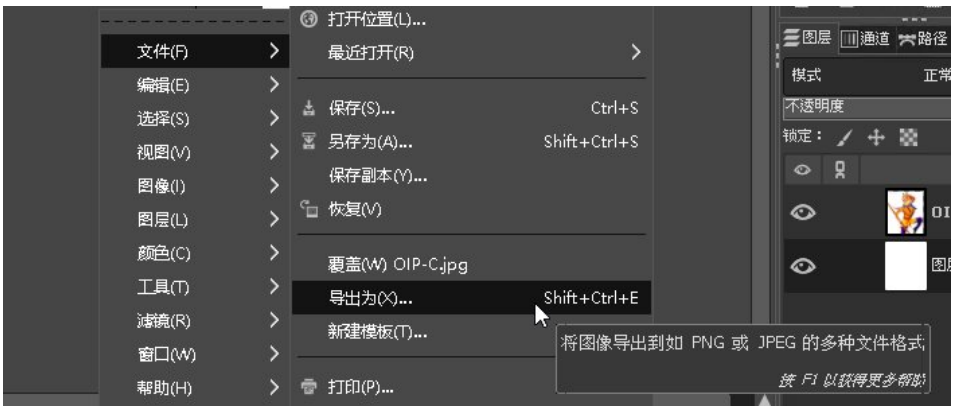
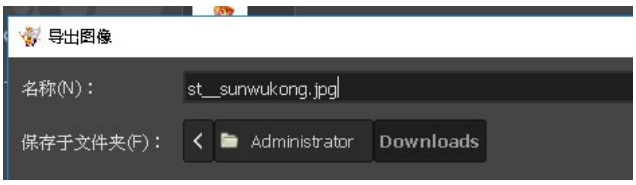


图 1: 毫无疑问，要缩放成 250x292 px

大功告成！导出图片吧。右键文件-> 导出为……



注意文件名应该和我们武将的命名一致。没忘记我们给武将取了啥内部名字吧？

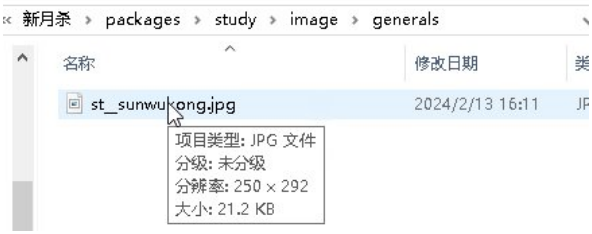


点击导出，这样就到了我们导出的最后一关了。我们把质量设为 80，这样尺寸和画质都处于比较均衡的水平。

36.4 将图片加入游戏

现在图片也准备就绪了！我们最后一步就是把图片放入拓展包中。

图片的位置上面提到过了，是拓展包下面的 image/generals 下面。毫无疑问，现在我们包里面还没有这样的文件夹，没有就新建！最终效果如下：



打开游戏！进入武将一览查看详情！

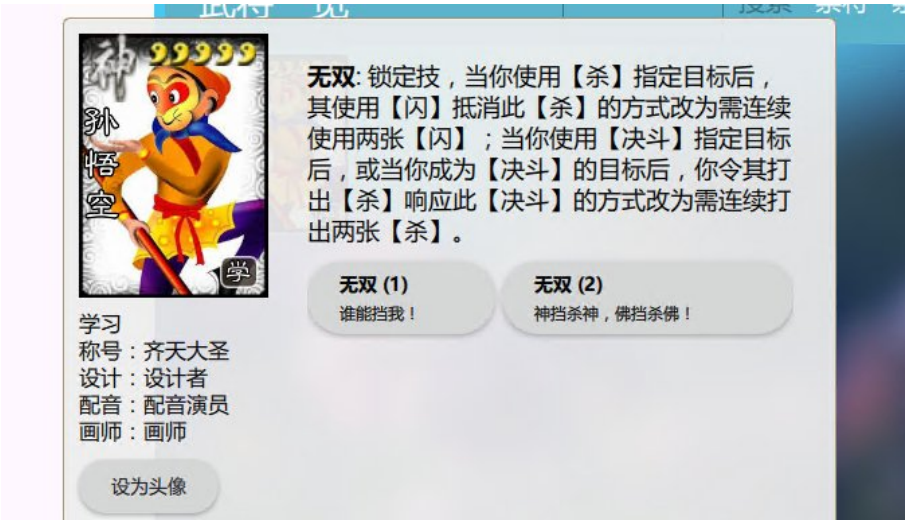


图 2: 好，很好，非常好

要不再点一下设为头像看看效果：

图 3: 小头像哦

CHAPTER 37

创建一个新技能

重头戏来了！

前面我们就知道了，武将的游戏体验和 TA 的技能如何有着好大好大的关系。怎么样？五体力上限带无双的孙悟空是不是比五体力上限的白板孙悟空玩起来带劲儿多了？

现在我们要继续强化这种体验感了！打造真正拥有自己个性的新技能！

听说周瑜因为长得帅所以能多摸牌？我们美猴王孙悟空怎么样？摸死你！

现在打开刚刚遗忘在角落里的 xuexi.lua 吧，要写代码了 ~

先把一份英姿的代码拷贝到我们的扩展包文件里面去吧。

```
local sunwukong = General:new(extension, "st__sunwukong", "god", 5)

local yingzi = fk.CreateTriggerSkill{
  name = "yingzi",
  anim_type = "drawcard",
  events = {fk.DrawNCards},
  on_use = function(self, event, target, player, data)
    data.n = data.n + 1
  end,
}
```

注意定义孙悟空的那句话一开始就有了，只是演示该粘贴的位置哦。

什么？哪里搞到的？当然是全局搜索咯。新月杀已有的技能也全都是用 Lua 语言写的，我们稍微搜搜技能内部名称自然就找到它们的 Lua 代码了。

言归正传，孙悟空那头都快上火了。

周瑜英姿多摸了一张牌，看到代码里面那个数字”+1”了吧？

```
data.n = data.n + 1
```

孙大圣那可是”美”猴”王”啊，才多摸一张牌哪儿说得过去啊？在后面加个零，多摸十张！

```
data.n = data.n + 10
```

行了，满意了。

核心代码已经完成，不过还有点需要修饰的工作要做。

技能名字要改改吧？重名的话，小心启动不了游戏哦。

把整个技能代码段选中，CTRL+H 把替换窗口叫出来。（在右上角那里呢）原来的技能叫什么来着……yingzi，嗯就是它，写在第一栏。再起个新名字，就叫 st__meiwang 好了，写在第二栏那里。

点击全部替换，名字就改掉了！

然后是翻译（还记得那个 Fk:loadTranslationTable 吧?）：

```
["st__meiwang"] = "美王",
[":st__meiwang"] = "摸牌阶段，你可以多摸十张牌。",
```

翻译一个技能是有一些要注意的地方的。

翻译技能的通式是这样的：

```
["技能名字"] = "技能名字的译文",
[":技能名字"] = "技能的描述",
```

技能名字的译文不要超过两个汉字，否则在游戏界面里面会显示的很奇怪……

技能名字前面加冒号，表示的是技能的描述。这个描述文本中可以加上 HTML 标签以制造某些文字效果，就跟网页代码一样。比如用 和 将一段文本夹起来，这段文本就被加粗了，要知道字体加粗是可以来显示锁定技、限定技、觉醒技之类的提示语的。

提示： 不过在实操中，我们不要对锁定技之类的提示语手动加粗。

现在技能已经写好了，剩下的只需要把这个技能添加到武将身上就行了。没错，还是用我们已经熟悉的 `addSkill` 函数，不过这次不用字符串形式了。这是因为技能对象 `st__meiwang` 是在同一个文件中创建好了的，不用引用别人的恩赐了！

```
sunwukong:addSkill("wushuang")
sunwukong:addSkill(st__meiwang)
```

技能就添加完成了！赶快到游戏里面体验这个拥有强力摸牌技能的齐天大圣孙悟空吧！

CHAPTER 38

添加音效

作为一个全方位立体化的武将扩展行为，音效的搅和也是必要的吧？耳旁还回荡着那动人的“犯大吴疆土者，盛必击而破之！”我们的新武将怎么能不来跟个风呢？

38.1 打造自己的声音工作室

新月杀的声音文件是 mp3 格式的，保存在拓展包的 audio 文件夹下。所以，别拿着什么 wav、wma、mp3 之类的干瞪眼了，试一试格式工厂吧。

严格地说，我们还需要拥有一个音频处理软件，用来进行可能的音频截取、音频合并、音频修整等操作；如果想使用自己的声音，还需要一个录音软件，甚至硬件上的配合（比如话筒等）。这里推荐开源音频处理软件 Audacity 哦。

好的，这样基本上我们的声音工作室就成型了。

38.2 添加技能使用音效

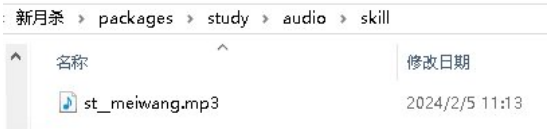
之前在武将一览里面，我们新技能“美王”根本没显示配音播放按钮呢。这就是因为没有对应的配音文件啊。这就去给他补上！

从网上找一段孙悟空的音频，下载之，或者利用 Audacity 进行电脑内录。然后继续扔到 Audacity 里面裁剪成我们需要的样子，”导出->mp3”，这样得到了处理后的 mp3 格式的音频，大功告成。（编者：其实我们是来炫耀肺活量的……）

提示： 为了不让加入的配音文件体积过大，请将码率设为 96Kbps~128Kbps 左右。在拓展包中占体积大头的就是语音，控制好体积还是需要注意的。

不管怎么说吧，反正现在是得到了 mp3 的配音文件，但是！文件的名字还没改嘛。先想想技能的名字叫什么来着？嗯，是 st___meiwang，所以配音文件的名字也就可以确定了，同样是：st___meiwang。（编者：团结一致向前看！）

接下来要做的就是把配音文件放到 audio 文件夹里面去了。但是！可不是直接放到 audio 文件夹下，而是放到里面那个 skill 文件夹下，因为我们现在做的是技能配音文件。



给技能配音文件起名字的通用方法是：

- 技能名字
- 技能名字＋编号

比如刚刚我们就是用的第一个方法。st___meiwang.mp3。

至于第二个方法，是当我们觉得这个技能需要分情况说台词的时候，或者就是有多个配音文件的时候，为了统一管理才用的。

看看这个游戏中绝大多数的技能都是有两句语音吧？一句多单调啊。

当使用第二个方法在技能名字后面添加编号的时候，在界面上相应地也会显示一个编号。

总之我们已经把音效放到正确的位置了，打开游戏体验吧。等会，怎么界面上没显示台词呢？原来还是忘记写翻译了呢。

打开我们的拓展包 xuexi.lua，在翻译表把台词加上！

```
["$st__meiwang"] = "技能 美王 的台词哦。",
```

台词的通用写法就是这样，用美元的那个符号引导：

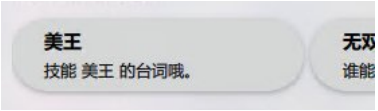
```
["$技能名称"] = "台词译文",
```

或者说是：

```
["$技能名称与编号"] = "台词译文",
```

用于翻译那类带编号的技能配音。（孙悟空：有什么了不起的啊？不就是用配音文件的文件名嘛……）

写好后重新启动游戏，这下终于看到亲切的台词了：



搞定！

38.3 添加武将阵亡音效

“霸业未成、未成啊……”

武将们总会有离开游戏的那一刻的，还是要给他们一个抒发壮志未酬满腔遗憾真不科学的感情的机会的啊。现在我们来为我们的新武将添加一个阵亡音效好吧。

其实和技能音效类似了，依然是各种选材、剪辑、格式转换工作，最后得到了我们需要的一份 mp3 格式的阵亡配音文件。

但是，文件名呢？毕竟不是技能配音啊，哪有技能名字可用啊？

其实，武将阵亡的配音文件的文件名，就是这个武将的名称了！所以我们现在做好的这份配音文件，就是 st___sunwukong.mp3！

把配音文件放到拓展包下的 audio/death/文件夹中，阵亡配音也就有了 ~

等等，好像还少点什么事情没做……嗯，是台词翻译！阵亡了也要有那么几句话不吐不快嘛。接着打开扩展包文件 study.lua，拉到最下面，找到翻译表，加上一句：

```
["~st__sunwukong"] = "不！还不可以死……",
```

（孙悟空：%@ &#%&%# @……）

算了还是按照实际情况写吧！总之这句话通常意义上是这么写的：

```
["~武将名称"] = "武将阵亡台词译文",
```

最前面那个小波浪”~”，代表相反的意思，生的反义词自然就是死啦，所以这样就表示武将的阵亡台词了。（听说人家程序员叫：析构）

不用管那么多了，保存一下，赶紧到游戏里面看看效果吧！

啊，还有一件事。如果是自己配的音，不要忘了把前面那条翻译：`["cv：武将名称"] = "配音人员"`，里面的配音人员改成自己的名字哦 ~

解决！

CHAPTER 39

发布与更新自己的拓展吧

到了这一步，想必你通过阅读教程或者直接参考现有代码，自己的拓展已经积累了不少内容了吧？既然如此，为何不分享出去让大家都看到呢？因此本节将介绍发布拓展的方法。

要发布拓展，步骤很简单：将拓展建立成 Git 仓库，并托管到 Gitee 等平台即可。这个对于经验充足的人而言非常简单，但对于连 Git 都没听说过的人还是有相当难度的——无妨，我们接下来慢慢的介绍就行。

39.1 关于 Git

Git 是众多版本控制工具之一。版本控制是一种记录一个或若干文件内容变化，以便将来查阅特定版本修订情况的系统。大体上来说，他是使用一个本地数据库管理不同文件的各个版本。当然，作为初来乍到的新月杀拓展学习者，你只要知道 Git 是我们用来发布和更新拓展的工具就行啦。

如果你之前接触过太阳神三国杀拓展开发的话，你可能用这种方式管理和发布拓展：直接复制一份出来，把文件夹改成拓展包-X.X，这就是新版本；然后打个包并发到网上。这种方法固然简单直观，但也存在几个明显的问题：

- 更新不便，每更新一下就要把整个拓展都重新下载一遍
- 复制了太多文件夹出来，占了太多地方
- 其他玩家更新拓展的时候操作不便

第一个问题往往是那些大体量拓展（>30MB 左右）的痛点，因此拓展开发者后来又想出了“补丁更新”的形式——先让玩家下载某个版本安装着，自己更新接下来几个版本的时候，就只发一个小补丁，这样玩家的下载量就减轻了。但是这样一来，当版本更新了几次之后，补丁的制作难度也越来越大——文件太多太杂啦！

所以我们一开始制作新月杀的时候选择 Git 来更新拓展包就是为了解决以上的几个问题。只要你在游玩新月杀，就该体验到更新的便捷性了：每次进服务器都会先自动更新一下拓展包，然后就能加入了，只要不是第一次下载的话更新也用不了多长时间。而且开发者的操作也方便多了，从来不需要向群文件（或者诸如网盘之类的）里面发打包好的 zip 包。

事实上，这样的自动更新功能就是 Git 实现的。下面我们就来正式接触 Git 吧。

39.2 安装 Git

前往https://git-scm.com/download/win下载 git，下载 64-bit Git for Windows Setup。这样应该会为您下载一个 exe 安装包。

考虑到官网的下载链接实际上指向 github，而且可能连官网的都进不去，所以也可考虑https://mirrors.tuna.tsinghua.edu.cn/github-release/git-for-windows/git/。

Git 的安装流程就是不断无脑的点下一步了，下一步下一步下一步……除了这里：

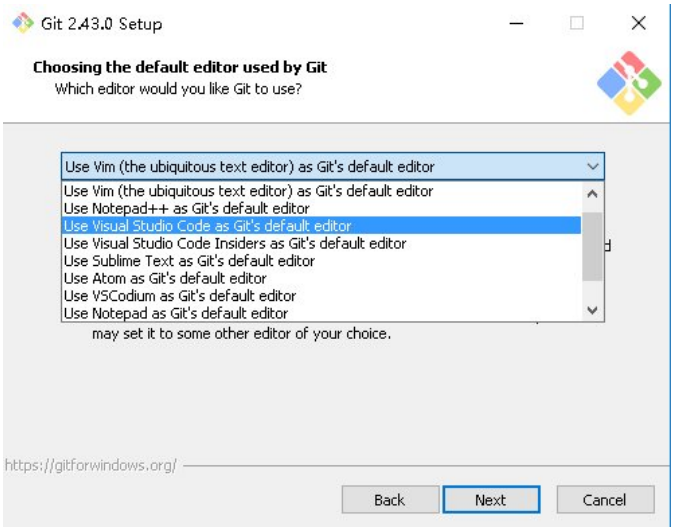


图 1: 修改默认编辑器为 vscode

这里默认会使用 Vim，切记改成 Visual Studio Code 再点下一步！

在各种下一步之后，安装就完成了。在桌面上右键单击，如果右键菜单有 Git bash here 的话，就说明安装成功啦。

虽然网上流传着许许多多的 Git 图形化客户端，但是本文将只会介绍 Git 命令行的使用！这是因为只有在命令行模式下你才能执行 Git 的**所有**命令，而大多数的 GUI 软件为了降低难度，只会让你能执行一小部分操作。如果你学会了在命令行下如何操作，那么你在操作 GUI 软件时自然也不会遇到什么困难。但是反过来就不成立了。

点开 Git bash here 就是开启命令行了。什么你从来没接触过命令行？那么赶紧去看《新月杀新人入门》的第 11 章吧。虽然题目说的是 Linux 的使用，但是对 Git bash 是同样适用的！（除了 Git bash 不可以用 apt 命令）在 Git bash 里面对着那一章操练一下，熟悉命令行的基本操作以及几个基本命令（ls、pwd、cd、mv 啥的）再来接着阅读后文吧。

备注： 在 Git bash 中，所谓的“家目录”指的就是 C:Users< 你的用户名 >。由于种种机缘巧合，Windows 版的 Git 没有中文版，你得去学着看懂那些英文才行。

39.3 初次运行 Git

熟悉了一下命令行的操作之后，我们便可以好好接触 git 本身了。

在随便某个文件管理器或者桌面右键->Git bash here 就能打开终端了……下面我们来做一些初次运行的配置吧。我们的目标是发布拓展，也就是上传到 Gitee 中，所以我们需要先让 Gitee 知道我们这台电脑对应着自己的那个账号才行，不然 git 命令行不好操作啦。

备注： 可能有些人会反感 gitee，但对于发布拓展而言这似乎是为数不多的好选择了。你也可以发布到 Github，如果你能保证所有游玩你拓展的人全都能顺利连上 github 的话。

好，开始吧，首先第一件事是告诉本机的 git 你是谁：

```
$ git config --global user.email "你的Git邮箱"
$ git config --global user.name "你的Git用户名"
```

这两行命令之后就告诉了 git 你自己的身份了。这里运行的程序名是 git；参数 1“config”表示要做的 git 操作类型是自定义配置，选项“–global”表示全局有效，参数 3 表示要配置的配置项，最后一个参数就是配置的值了。

接下来就是要告诉 Gitee，自己这台电脑可以给自己的账户用了。Git 采取的是 SSH Key 的认证策略，所以我们紧接着要来配置。

提示： SSH Key 更具体一点是采用 RSA 加密来进行认证的。RSA 加密是一种常见的非对称加密方式，你自己手上会有一个“公钥”和“私钥”，私钥自己保留，公钥可以告诉任何人。因为必须要知道私钥内容才能认为公钥有效，所以只要你手上有私钥，就起到认证身份的作用了。更多信息可以自己去网上查询到。

我们先在命令行中敲以下这些命令，来创建好自己的一对密钥：

```
$ ssh-keygen -t rsa -C "你注册用的邮箱地址" # 换成自己真正的邮箱
  # 出来一堆东西，一路点回车就是了
$ cat ~/.ssh/id_rsa.pub
  # 出来一堆乱七八糟的东西：ssh-rsa <一大堆乱七八糟的内容> <你的邮箱>
```

注意井号开头的表示注释；没错，命令行也可以写注释的，毕竟命令行本身也是一种脚本语言（我们用不到它的编程功能就是了）。总之输命令的时候无需输入井号之后的内容。

在命令 `cat ~/.ssh/id_rsa.pub` 中，出来的那一堆以 ssh-rsa 开头的输出，就是我们刚刚创建好的公钥。只要把公钥告诉 gitee 就能证明自己的身份了。

进入 Gitee 网页，然后如下操作：

1. 点右上角你的头像，点账号设置
2. 点左侧栏中安全设置 - SSH 公钥
3. 此时弹出公钥添加界面，标题任选，下面公钥栏中，将刚刚生成的公钥复制粘贴上去
4. 点确定

这样就配置好了 ssh 公钥。我们还是验证一下配置是否成功吧，使用命令：

```
$ ssh -T git@gitee.com
Hi xxxx! You've successfully authenticated, but GITEE.COM does not provide␣
↪shell access.
```

输出像 Hi xxx! 这样的信息，就说明配置成功了。否则需要进一步检查自己的操作，上网查一下吧。

到这一步为止，你已经完成了 git 的初步配置，现在你已经有能力将拓展包发布出去了。

39.4 为拓展包创建 git 仓库

用文件管理器打开自己正在制作的那个拓展包的文件夹，右键 Git bash here 打开命令行。

图 2: 打开 Git bash

图 3: Git bash 的界面

从命令提示符中可以看到我们现在的当前目录就是拓展包的文件夹。不过此时这里还建立 Git 仓库，所以我们要使用如下命令先创建一个空白的仓库：

```
$ git init
```

在这个命令中，运行的程序依然是 git，操作类型是 init，表示新建空仓库。（或称为初始化仓库，反正都一个意思）

可是我已经写了这么多代码了，为什么新建的仓库却是空的呢？明明已经有这么多文件了呀。为了弄明白这些，我们需要先知道 Git 的三种状态：

- 已修改（modified）：表示修改了文件，但还没保存到 Git 数据库中。
- 已暂存（staged）：表示对一个已修改文件的当前版本做了标记，使之包含在下次提交的内容中。
- 已提交（commit）：表示数据已经安全地保存在本地 Git 数据库中。

Git 数据库又是个啥呢？大致的说，这是保存着仓库所有文件及其修改记录的地方。你对文件做出的种种修改、新增、删除，Git 都给你妥善保管者，前提是你提交了那些修改。在运行完 `git init` 命令之后，你的拓展包文件夹里面就会出现一个.git 文件夹，那是个隐藏文件夹，里面的内容就是 Git 为你保存的数据。

关于所谓的暂存啊提交啊三言两语很难讲明白，还是得从实践着手。既然现在我们已经创建好空仓库了，下一步我们就应该让 Git 追踪这些文件并创建初始提交了。使用命令：

```
$ git add .  # 将文件夹中所有的文件都加入暂存区
$ git commit -m "init" # 提交目前所有的文件，这样文件就正式存在于仓库里面了
```

在 `git add .` 中，那个点表示的就是“当前目录”。回忆一下前面所说的，点点表示的是上级目录。add 的作用就是将这些文件加入暂存区。而 commit 意为提交，选项`-m` 表示提交信息（你得用一句话表示这个提交是干啥的），这里随便写了一句 init 表示这是本仓库的第一次提交。

这样一来，初始提交也完成了，现在我们已经有了一个真实项目的 Git 仓库了，并从仓库里面检出了所有文件的**工作副本**。（所谓检出就是从 Git 数据库中拿出那个文件，仓库中所有文件都是在 Git 管理范围内的）通常，你会对这些文件做些修改，每当完成了一个阶段的目标，想要将记录下它时，就将它提交到仓库。

请记住，你工作目录下的每个文件都不外乎这两种状态：**已跟踪**或**未跟踪**。已跟踪的文件是指那些被纳入了版本控制的文件，在上一次快照中有它们的记录，在工作一段时间后，它们的状态可能是未修改，已修改或已放入暂存区。简而言之，已跟踪的文件就是 Git 已经知道的文件。

因为我们这里暂存了所有文件并把它们都提交了，所以现在所有的文件都处于已跟踪状态。我们还是继续完成最初的目的吧，将自己的拓展发到 gitee 上去才好。

39.5 上传拓展

要将新仓库发到 gitee 去，你首先得在网页里面新建一个仓库。我们现在用的这个仓库名叫 study，那就创建一个名为 study 的仓库吧。

图 4: 在页面右上角找到新建仓库

图 5: 创建与文件夹同名的仓库

注意下面初始化仓库啥的都不要勾选：

图 6: 不勾选那三个初始化的框

这样一来，就在网页里面成功为自己账号创立了一个空仓库了。接下来要做的就是将自己的**本地**仓库推送到**远程仓库**去。

网页里面应该已经告诉你如何操作命令行了，不过现在还是按照文章说的步骤来吧。

刚刚在网页中做的一系列操作就是创建了一个远程仓库。远程仓库存在的意义就是和他人进行协作，当然了备份也算个功能。不过我们的本地仓库还不知道自己已经有远程仓库了，现在我们就告诉他：

```
$ git remote add origin git@gitee.com:你的用户名/study
```

这样一来，我们就创建了一个名叫 “origin” 的远程仓库（远端），他的 url 是后面那个。有了远程仓库的话，我们就可以进行推送与拉取了。推送就是将本地的内容发送到远程仓库去，拉取则反之。顺便，推送是 push，拉取则是 pull。至于远程仓库的名字为何是 origin，这是因为 `git clone` 的时候默认会给远程仓库取这个名字。

接下来就将我们的 master 分支推送到远程仓库吧：

```
$ git push
```

什么，推送失败了？这是因为我们现在还是第一次推送，`git push` 命令不能用在第一次。第一次推送的话需要用这个命令：

```
$ git push -u origin master
```

这个命令的作用就是把本地的 master 分支和远端 origin 的 master 分支联系起来，以后就可以直接用 git push 命令了。

推送完成之后，刷新一下 gitee 的网页，你就发现文件都已经发上来了。快把仓库链接分享出来吧！（当然 study 就算了，可以多多分享自己的 DIY 仓库）

39.6 更新拓展

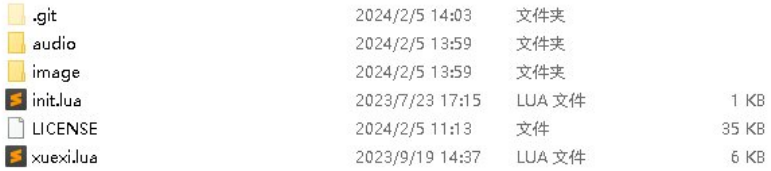
又回到更新这个话题上来了。欲更新拓展，需要走如下几步：

1. 修改已有的文件，这样才有东西可以更新嘛
2. 用 `git add .` 命令暂存所有文件，这样自然就包含了要更新的文件
3. 用 `git commit -m "xxx"` 命令创建好提交
4. 用 `git push` 命令完成推送，至此完成更新

还是举个例子为好。我们现在做的拓展也已经属于万千开源项目之一了，开源项目自然要有自己的许可证。因为新月杀本身是 GPLv3 协议开源的，所以拓展也需要用这个协议开源（协议里面规定了），声明协议的方式就是复制一个 LICENSE 文件进来。

新月杀本体里面已经包含一个 LICENSE 了，把他复制到拓展包文件夹里面吧。

复制了 LICENSE 文件之后文件夹内长这样：



.git	2024/2/5 14:03	文件夹	
audio	2024/2/5 13:59	文件夹	
image	2024/2/5 13:59	文件夹	
init.lua	2023/7/23 17:15	LUA 文件	1 KB
LICENSE	2024/2/5 11:13	文件	35 KB
xuexi.lua	2023/9/19 14:37	LUA 文件	6 KB

图 7: 复制了 LICENSE 之后

然后在 git bash 操作吧：

这样一来，我们就成功提交了最新的改动，并且推送到了 Gitee。这就是拓展更新的步骤。

CHAPTER 40

添加提示信息

游戏的互动性很强大，我们新月杀自然也不例外。不仅仅是游戏界面，甚至还包括代码层次上的互动呢！

40.1 添加代码注释

在 Lua 文件中，代码注释主要有两种，单行注释和区块注释。

单行注释的符号是英文状态下的两个减号 `--` ，本行从这个符号开始，向后的所有内容都算作注释文本了。

```
local extension = Package:new("xuexi")
-- 我是单行注释 ^_^
extension.extensionName = "study"
```

区块注释的符号则比较复杂，像括号一样分为两个部分。

前半部分是英文状态下的两个减号和两个左侧的方括号 `--[[` ；后半部分呢，就是两个右侧的方括号 `]]` 了。

被这两个部分括起来的文本，就都算作注释文本了。

悄悄地记住：被注释的文本是会被新月杀杀忽略掉的，所以可以在注释里面随便写东西喽！

还有一个小窍门：区块注释可以用来封杀一段 Lua 代码，不让它们出现在游戏中。以后心情好了还能取消注释，这样就不用直接删除某些代码了。

使用了注释的话，被注释的文本会被涂抹成不显眼的颜色，看上去其实还是挺养眼的……

注释有什么用处呢？答案：交流。

```
--[[
  拓展包名称：学习包（xuexi）
  说明：这个拓展包是我学习武将拓展用的
  作者：notify
  武将数目：总共1名武将，已完成1名武将
  武将列表：
    孙悟空（st__sunwukong）
  尚需完善的内容：好多好多……
  备注：
    百度新月杀吧欢迎您！
--]]
```

可以注明代码的作者、功能、版本信息；

可以注明代码的参数要求如何、效果是什么；

可以注明代码的设计思路；

可以注明代码中还存在的问题、注明还需要写些什么；

可以注明代码的各种各样的细节信息……

甚至可以写上一句“写完这一段要记得去吃泡面哦 ^_^”。

总之是……很有用的（培养办公室恋情的必备工具 ~）。

40.2 显示战报信息

如果说代码注释是写游戏的人们之间的交流，那么提示信息就让我们玩游戏的人也加入了这个温馨的圈子。

说到提示信息，首先应该认识一下新月杀为我们提供的一个信息结构体：`LogMessage` 。

```
-- 见于lua/server/system_enum.lua
---@class LogMessage
---@field public type string
---@field public from? integer
---@field public to? integer[]
---@field public card? integer[]
---@field public arg? any
---@field public arg2? any
---@field public arg3? any
```

从中，我们可以看出 `LogMessage` 由以下一些参数构成：

1. `type`: 信息的输出类型
2. `from`: 涉及的行为发起者
3. `to`: 涉及的所有行为接受者列表
4. `card`: 可能涉及的卡牌信息

5. arg, arg2, arg3: 可能涉及的参数，最多带三个

首先，要显示提示信息，应先创建一条消息，也就是要产生一个 LogMessage 结构体的实际对象。对应的代码是这样的：

```
local msg = {} -- 创建一个LogMessage
```

通过这条代码，我们定义并创建了一个消息，名字叫 msg。

提示： 什么这不就只是一张表而已吗！确实是。Lua 中万事万物都是表，只是我们给某些表赋予了特殊的意义而已。

然后呢？想想我们要向玩游戏的人说什么话。比如说：

"Hello！新月杀向你孙悟空问好！"

在这句话里，我们表达的是一个问好的行为，而孙悟空则是这个行为的接受对象。因此对应到 LogMessage 的那些参数，可以想到：to= 孙悟空。

而且我们也顺便就定义了这个提示信息的输出结构：Hello！新月杀向你%to 问好！

这里的百分号"%"表示接下来的内容是要被对应的参数替换掉的，所以到时候新月杀一看到这个%，就会把那个 to 换成我们设定的值，孙悟空。

现在我们同时还明白了那个 type 参数的含义。不多说了，赶紧在代码中定义这样的一个 type 吧。

把目光转向那个已经有点无所不能的样子的翻译表吧，在这里写上我们的输出结构：

```
["#hello"] = "Hello! 新月杀向你 %to 问好! ",
```

在这里，我们定义的消息输出结构类型的名字叫做 "#hello" 。

值得一提的是，在定义信息输出结构的时候，推荐使用引导符号"#"作为输出结构名字的一部分。

现在可以填充 LogMessage 结构体了。由于这个输出结构中只涉及到了参数 to ，并没有涉及所有的参数，所以没有涉及的那些就可以不管了。

```
on_use = function(self, event, target, player, data)
  local msg = {} -- 创建一个LogMessage
  msg.type = "#hello"
  msg.to = { player.id }

  data.n = data.n + 10
```

还记得好早之前提过的一个问题吗？

一个对象调用自己的成员函数的时候，中间用冒号":"连接；

这里我们又遇到类似的事情了，请注意 msg 和 type 之间的那个小点"."

要知道 type 可不是什么成员函数，只是 LogMessage 结构体中的一个参数而已。所以这次我们不用冒号了，用小点（也就是英文的句号）连接这个对象和它的参数。

这种结构体中的参数，这种用来表现对象的各个属性和状态的参量，被称作对象（或结构体）的成员变量。

又默默地记下了一句话：一个对象调用自己的成员变量的时候，中间用点”.”连接。

目光向下移动……`{ player.id }`？为什么这个不直接写 id 而要套层壳了？

其实在 `LogMessage` 的介绍里面已经知道了，to 的值是一个列表 `integer[]` ，是所有接受者的列表。一个列表和一个接受对象怎么能等同呢？级别明显不够嘛！所以才要套个花括号壳，表明这是一个列表才行。（回想之前的 Lua：列表章节）

本来以为没问题了，结果又引出了一个 `player` ……这是？

目光向上 ~

```
on_use = function(self, event, target, player, data)
```

在那个 function 的括号里面，有五个参数 `self` ，`event` ，`target` ，`player` 和 `data` 。

我们需要的 `player` 就是其中的第三个参数了。

它是一个角色对象，也就是传说的 `ServerPlayer` ，是玩游戏的人在新月杀中的代表。这个 `player` 里面包含了许多玩游戏的人的信息，比如在哪个游戏房间啦，周围挨着的有哪些角色啦，有哪些手牌啦什么的。

有关 `ServerPlayer` 的具体信息可以从 lua/server/serverplayer.lua 中找到，这里就先不管它了。

OK，现在消息已经被我们填好了，接下来要做的，是把这条消息发送到游戏中去，让玩游戏的人能看到它。看法宝！哦……不是，应该说：看代码！

```
player.room:sendLog(msg)
```

我们的消息就是通过这么一句轻轻松松地发送到了游戏里面……

等等，`room` 是什么？是游戏房间啦，房间 ~

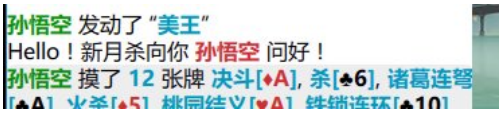
玩游戏怎么也要有场地吧？别告诉我真是在月亮上面玩的……

`room` 就是这样一个表示游戏场所的房间对象，而它的真身是传说中的 `Room` 老兄。在 `Room` 这里保存了一场游戏里面的很多信息，比如游戏模式啦，还有哪些角色存活啦，该谁行动啦什么的。

`room` 这个对象很重要，许多扩展行为都跟它有密切关系的，因为它可以帮我们做许多事情，比如现在就是它在帮我们发送这条消息的。事实上，大家摸牌、弃牌啥的操作都是 room 在帮忙完成哦，想想自己是不是点点确定就够了？room 就是充当着这样一个发牌员的角色，绝大多数操作都是它完成的。几乎每次扩展，都要想办法先把它鼓捣出来，可真要和它搞好关系哦 ~

这不，我们就通过 `player.room` 搞出来了这个角色所在的房间，然后就开始操作啦。

好了不管那么多了，快到游戏里面看看我们发送的消息吧。当孙悟空发动美王技能的时候……



成功!

CHAPTER 41

修改距离

军争时代，节奏加快，距离的重要性大大加强了。对于一个跟头十万八千里的孙悟空来说，当然不能忍受落在他人的后面！筋斗云，我们一起去代码里面讨个公道……

好吧，那么这次我们就要为我们的新武将创建这样一种可以用来修改距离的技能。

先把马术的代码拿来研究一下：

```
local mashu = fk.CreateDistanceSkill{
  name = "mashu",
  frequency = Skill.Compulsory,
  correct_func = function(self, from, to)
    if from:hasSkill(self) then
      return -1
    end
  end,
}
```

可以看到，创建距离修改技能需要用到一个叫做 `fk.CreateDistanceSkill` 的函数，其原型是：

```
 fk.CreateDistanceSkill{
  name = "xxx",
  frequency = xxx,
  correct_func = xxx,
}
```

这个方法有三个参数，name, frequency 和 correct_func。

- `name`: 表示这个距离修改技能的名字；
- `frequency`: 技能的标签。一般来说修改距离的技能都是锁定技啦，填入 `Skill.Compulsory` 就表示是锁定技了。

- correct_func: 是一个规定距离修改标准的函数，告诉电脑具体怎样计算距离。

其中这个 correct_func 函数的原型是: function(self, from, to) 它需要三个参数，分别是 self, from 和 to。

- self：就是技能本身啦；
- from：是位于距离计算的起点处的那个角色。因为距离计算是有方向性的呀，当然需要一个起点角色，还有一个终点角色了。
- to：不用说了，这就应该是位于距离计算的终点处的角色了。

因为距离计算的特性，距离修改技能的效果是对距离计算的双方都有影响的，所以即使我们本身没有这个技能，计算距离时也要考虑对方是否有类似的技能。故而 correct_func 函数中提供了两个角色对象 from 和 to ，用来考虑这两方面的计算情形。

对于马术来讲，它的描述是“你计算与其他角色的距离-1”，所以当拥有马术的我们处于起点，其他角色作为终点的时候，马术就会起作用。而当我们自己作为终点，其它角色处于起点的时候，即使我们拥有马术，也没有效果。

所以马术的代码里面，只需要考察 from 是否拥有马术这个技能，进行距离修正就可以了。可以想到，飞影的代码里，应该也只需要考察 to 就可以了。而义从的代码里面则要 from 和 to 两方面都要考虑才行。

现在我们要为孙悟空设计距离修改技能了。想法是这样的：

“腾云：你计算的与其他角色的距离-5；其他角色计算的与你的距离 +5。”

代码如下：

```
local tengyun = fk.CreateDistanceSkill{
  name = "st__tengyun",
  correct_func = function(self, from, to)
    if from:hasSkill(self) then
      return -5
    end
    if to:hasSkill(self) then
      return 5
    end
  end,
}
```

别忘了用 addSkill 让悟空学会该技能！

好了，赶紧体验一下这种高耸入云无人能及的感觉吧！

悄悄地告诉自己：按下 D 键可以查看到他人的距离。

我们来实操一下，开桌军八，可以看到到所有人距离都是 1 了。

不过怎么看别人到我的呢？这里就轮到我们重要的测试工具人——谋徐盛出场啦。

开 2 人身份局，点将谋徐盛（测试包中，或者直接用搜索功能）。然后发动技能“变更”将对方变成孙悟空，接下来按下 D 键查看距离：

至于设计实现类似“与某名角色的距离始终为 1”的效果，`CreateDistanceSkill` 其实还提供了 `fixed_func` 这个参数，他和 `correct_func` 类似，只是把之间的距离设为了固定值，这个就请自己探索吧！

Part VI

代码架构解析